P1.T09.01.02. Торка: Пример: Разломак

1. Тип података: Торка

Торка (Tuple) је структура података која омогућава да се у једној целини чува више вредности, које могу бити различитих типова. На пример, једна торка може да садржи: цео број, реалан број и ниску знакова.

Торке су корисне када желимо да **групишемо неколико повезаних података**, а да притом **не морамо да дефинишемо посебну класу или структуру**. Ако желимо да објединимо неколико података истог типа, тада уместо дефинисања новог структурног типа можемо употребити библиотечки тип торки.

У математици се често помињу **уређени парови** и **уређене n-торке**.

Иако и **C#** и **C++** подржавају торке, **начин њиховог коришћења се разликује**.

Торке у програмском језику C#

У језику C# n-торке се могу преставити типом **`Tuple`**.
На пример, тип **`Tuple<int, int, int>`** представља тројку целих бројева.

Торке се могу креирати помоћу функције **`Tuple.Create`**.
На пример, датум можемо представити торком тако да прво ставимо годину, па затим месец и на крају дан:

```
Tuple<int, int, int> datum = Tuple.Create(2026, 10, 19);
```

Поља торке се једноставно могу очитати (нпр. **`datum.Item1`**, **`datum.Item2`** и **`datum.Item3`**),
али се **не могу мењати**.

Основна предност торки у односу на структуре је то што се могу **једноставно лексикографски поредити**.

Торке се могу поредити методом **`CompareTo`** која враћа:

- **-1, негативан резултат** ако је торка на којој је та метода позвана **испред** торке наведене као аргумент,
- **0, нулу** ако су једнаке и
- **1, позитиван резултат** ако је **прва торка иза друге**.

Пре позива **`CompareTo`** потребно је извршити **експлицитну конверзију** у **`IComparable`**. На пример

```
if (((IComparable)datum1).CompareTo(datum2) < 0) // datum1 je ispred datum2
```

Торке у програмском језику C++

У језику C++ n-торке се могу преставити типом **`tuple`** дефинисаним у заглављу **`<tuple>`**.
На пример, тип **`tuple<int, int, int>`** представља тројку целих бројева.

Торке се могу иницијализовати помоћу витичастих заграда или креирати функцијом **`make_tuple()`**.
На пример, датум можемо представити торком тако да прво ставимо годину, па затим месец и на крају дан:

```
tuple<int, int, int> datum1 {2026, 10, 19};
tuple<int, int, int> datum2 = make_tuple(2011, 11, 30);
```

Поља торке се једноставно могу очитати функцијама **`get<i>`** где је са **i** означен редни број компоненте торке који желимо да прочитамо. На пример:

- **`get<0>(datum1)`** враћа вредност **2026**, а
- **`get<2>(datum2)`** вредност **30**.

Постоји и начин да се комплетна торка распакује на појединачне вредности, на пример позив:

```
tie(godina, mesec, dan) = datum1
```

би у променљиве **`godina`**, **`mesec`** и **`dan`** сместио вредности одговарајућих поља торке **`datum1`**.

Основна предност торки у односу на структуре је то што се могу **једноставно лексикографски поредити** коришћењем уобичајених релацијских оператора **<**, **<=**, **>** и **>=**.

Торке у програмском језику C# (наставак)

У језику C# постоје два важна начина за представљање торки:

1. **Tuple** — класични начин, доступан од старијих верзија .NET-а.
2. **ValueTuple** — савременији начин, уведен у C# 7.0, који омогућава једноставније записивање и именовање елемената.

Класична торка (Tuple) у програмском језику C#

Класична торка се креира помоћу методе **`Tuple.Create()`** или конструктора одговарајуће класе **Tuple**. Пример торке која садржи бројилац и именилац разломка:

```
using System;
class Program
{
    static void Main()
    {
        Tuple<int, int> razlomak = Tuple.Create(3, 4);
        Console.WriteLine(razlomak.Item1);
        Console.WriteLine(razlomak.Item2);
    }
}
```

Објашњење:

- **Tuple<int, int>** представља торку са два елемента типа **int**.
- **Tuple.Create(3, 4)** креира торку чији су елементи 3 и 4.
- **Item1** приступа првом елементу.
- **Item2** приступа другом елементу.

Класична торка може да садржи и елементе различитих типова. На пример:

```
Tuple<int, string, double> osoba =  Tuple.Create(25, "Marko", 182.5);
```

Ова торка садржи цео број, ниску знакова и реалан број.

Важно:

- Елементи класичне торке су непроменљиви након креирања.
- Ако желимо да променимо неку вредност, креирамо нову торку!

Савремена торка (ValueTuple) у програмском језику C#

Од верзије C# 7.0 могуће је користити једноставнији запис торке.

```
using System;
class Program
{
    static void Main()
    {
        (int, int) razlomak = (3, 4);
        Console.WriteLine(razlomak.Item1);
        Console.WriteLine(razlomak.Item2);
    }
}
```

Овај начин је краћи и прегледнији од класичног **Tuple**.

Још једна предност је могућност именовања елемената торке. На пример:

```
using System;
class Program
{
    static void Main()
    {
        var razlomak = (brojilac: 3, imenilac: 4);
        Console.WriteLine(razlomak.brojilac);
        Console.WriteLine(razlomak.imenilac);
    }
}
```

Уместо **Item1** и **Item2**,
користимо смислене називе **brojilac** и **imenilac**.

То чини програм лакшим за разумевање.

За разлику од класичног **Tuple**, елементи торке типа **ValueTuple** могу да се мењају:

```
var t = (x: 10, y: 20);
t.x = 15;
Console.WriteLine(t.x);  // 15
```

Враћање више вредности из методе

Једна од најкориснијих примена торки у C# јесте враћање више вредности из једне методе:

```
static (int, int) Razlomak()
{
    return (3, 4);
}
```

Метода враћа торку са два цела броја, а позивалац добија обе вредности

Именована торка може да учини намеру још јаснијом:

```
static (int brojilac, int imenilac) Razlomak()
{
    return (3, 4);
}
```

Овај приступ је нарочито користан када метода треба да врати резултат и додатну информацију о том резултату

Торке у програмском језику C++ (наставак)

У језику C++ торке су доступне преко библиотеке <tuple>. Стандардни тип се назива std::tuple.
За разлику од C#-а, где постоје класични Tuple и ValueTuple, у стандардном C++-у главни тип за ову намену је std::tuple:

Креирање торке:

```
#include <iostream>
#include <tuple>
using namespace std;
int main()
{
    tuple<int, int> razlomak = {3, 4};
    cout << get<0>(razlomak) << endl;
    cout << get<1>(razlomak) << endl;
    return 0;
}
```

Објашњење:

- **Tuple<int, int>** представља торку са два елемента типа **int**.
- **razlomak = {3, 4}** иницијализује торку чији су елементи 3 и 4.
- **get<0>(razlomak)** приступа првом елементу.
- **get<1>(razlomak)** приступа другом елементу.

Важно: у C++-у индексирање елемената торке почиње од нуле, као и код низова. У C# класичном Tuple-у први елемент је Item1, а у ValueTuple-у може да се користи име елемента.

Торка може да садржи елементе различитих типова. На пример:

```
tuple<int, string, double> osoba = {25, "Marko", 182.5};
```

За разлику од класичног C# типа Tuple, елементи торке у C++-у могу да се мењају:

```
tuple<int, int> t = {10, 20};
get<0>(t) = 15;
cout << get<0>(t) << endl;
```

Елементи се могу издвојити и у засебне променљиве помоћу std::tie:

```
int x, y;
tuple<int, int> t = {10, 20};
tie(x, y) = t;
```

У новијим верзијама C++-а може се користити и структурирано издвајање елемената, такозвани **structured bindings** (ова могућност је доступна од стандарда **C++17**):

```
tuple<int, int> t = {10, 20};
auto [x, y] = t;
```

Поређење торки у програмским језицима C# и C++

У језику C# n-торке се могу преставити типом **`Tuple`**.
На пример, тип **`Tuple<int, int, int>`** представља тројку целих бројева.

Особина	C#	C++
Основни тип	**`Tuple`** или **`ValueTuple`**	**`std::tuple`**
Библиотека	**`System`** / одговарајућа подршка за **`ValueTuple`**	**`<tuple>`**
Креирање	**`Tuple.Create(3, 4)`** или **`(3, 4)`**	**`tuple<int, int> t = {3, 4};`**
Приступ елементу	**`Item1`**, **`Item2`** или имена елемената	**`get<0>(t)`**, **`get<1>(t)`**
Индексирање	**`Item1`** почиње од првог елемента	**`get<0>`** почиње од нултог елемента
Различити типови	Да	Да
Промена елемената	**`Tuple`** — не; **`ValueTuple`** — да	Да, код обичне торке
Именовани елементи	Да, код **`ValueTuple`**	Не на исти начин; могу се користити **`struct`** или друга решења
Враћање више вредности из методе	Да	Да, односно из функције
Издвајање елемената	Деконструкција, нпр. **`var (x, y) = t;`**	**`std::tie`** или структурирано издвајање у C++17

Пример: Враћање разломка из функције

Упоредимо како би изгледала функција која враћа бројилац и именилац разломка:

C#

```
static (int brojilac, int imenilac) VratiRazlomak()
{
    return (3, 4);
}
var r = VratiRazlomak();              // Poziv:
Console.WriteLine(r.brojilac);
Console.WriteLine(r.imenilac);
```

C++

```
tuple<int, int> VratiRazlomak()
{
    return {3, 4};
}
auto r = VratiRazlomak();    // Poziv:
cout << get<0>(r) << endl;
cout << get<1>(r) << endl;
```

Обе функције враћају две вредности упаковане у једну целину.
Разлика је у томе што C# омогућава директан приступ именованим елементима торке, док се у наведеном C++ примеру елементима приступа помоћу **get**.

Пример 1: Задатак: Разломци

Напиши програм који сабира и множи разломке.

Улаз

Са стандардног улаза се уносе два разломка, сваки у посебном реду. Сваки разломак је записан тако што је су записани његов бројилац и именилац, раздвојени карактером **/**.

Излаз

На стандардни излаз исписати у првом реду збир, а у другом производ разломака. Разломци се исписују у истом формату у ком су учитани, максимално скраћени (бројилац и именилац су узајамно прости).
Пример

Пример

Улаз

2/3
1/4

Излаз

11/12
1/6

Решење

Коначна верзија програмског кода, односно решења задатка би била попут следећег:

C#

```
using System;
using Razlomak = System.Tuple<int, int>;
class R1_T09_02_Tuple_Razlomak
{
    static Razlomak ucitaj()
    {
        string[] s = Console.ReadLine().Split('/');
        int brojilac = int.Parse(s[0]);
        int imenilac = int.Parse(s[1]);
        Razlomak r = Tuple.Create(brojilac, imenilac);
        r = skrati(r);
        return r;
    }
    static void ispisi(Razlomak r) { Console.WriteLine(r.Item1 + "/" + r.Item2); }
    static Razlomak saberi(Razlomak r1, Razlomak r2)
    {
        Razlomak r = Tuple.Create(r1.Item1 * r2.Item2 + r1.Item2 * r2.Item1, r1.Item2 * r2.Item2);
        return skrati(r);
    }
    static Razlomak mnozi(Razlomak r1, Razlomak r2)
    {
        Razlomak r = Tuple.Create(r1.Item1 * r2.Item1, r1.Item2 * r2.Item2);
        return skrati(r);
    }
    static int NZD(int a, int b)
    {
        while (b != 0) { int p = a % b; a = b; b = p; }
        return a;
    }
    static Razlomak skrati(Razlomak r1)
    {
        int nzd = NZD(r1.Item1, r1.Item2);
        return Tuple.Create(r1.Item1 / nzd, r1.Item2 / nzd);;
    }
    static void Main()
    {
        Razlomak r1 = ucitaj();
        Razlomak r2 = ucitaj();
        ispisi(saberi(r1, r2));
        ispisi(mnozi(r1, r2));
    }
}
```

C++

```
#include <iostream>
#include <cmath>
#include <tuple>
using namespace std;
using Razlomak = tuple<int, int>;
int NZD(int a, int b)
{
    while (b != 0)
    {
        int tmp = a % b;
        a = b;
        b = tmp;
    }
    return a;
}
void skrati(Razlomak &r)
{
    int nzd = NZD(get<0>(r), get<1>(r));
    int br = get<0>(r) / nzd;
    int im = get<1>(r) / nzd;
    r = make_tuple(br, im);
    // return r;
}
Razlomak ucitaj()
{
    char c;
    int br, im;
    cin >> br >> c >> im;
    Razlomak r = make_tuple(br, im);
    skrati(r);
    return r;
}
void ispisi(const Razlomak &r) { cout << get<0>(r) << "/" << get<1>(r) << endl; }

Razlomak saberi(const Razlomak &a, const Razlomak &b)
{
    int br = get<0>(a) * get<1>(b) + get<1>(a) * get<0>(b);
    int im = get<1>(a) * get<1>(b);
    Razlomak r = make_tuple(br, im);
    skrati(r);
    return r;
}
Razlomak mnozi(const Razlomak &a, const Razlomak &b)
{
    int br = get<0>(a) * get<0>(b);
    int im = get<1>(a) * get<1>(b);
    Razlomak r = make_tuple(br, im);
    skrati(r);
    return r;
}
int main()
{
    Razlomak a = ucitaj();
    Razlomak b = ucitaj();
    ispisi(saberi(a, b));
    ispisi(mnozi(a, b));
    return 0;
}
```

Пожељно је да тестирамо извршавање програмског кода у развојном окружењу за дате улазне податке из текста задатка, односно за разломке чије су вредности **2/3** и **1/4**, али и за још неке тест примере, које ћемо сами припремити, као што су, на пример:

1/2 и **1/4**, **1/2** и **1/2**, **3/2** и **2/4**, **1/1** и **2/1**, **0/1** и **0/2**, **-1/5** и **2/-1**, **7/5** и **4/3** и сл.

2. Укратко (уместо закључка)

Као и сваки други задатак и овај можемо решити на неколико тачних начина, као на пример:

1. Коришћењем само **основних типова података** (`int`) за 4 улазне променљиве.
2. Коришћењем **структурног** (`struct`) типа података за разломак.
3. Коришћењем типа података за **уређене торке** (`Tuple`).
4. Коришћењем других типова података, као на пример **колекције** (низ, листа).

Када је корисно користити торке?

Торке су посебно корисне када:

- желимо да **вратимо више вредности из функције** или методе;
- представљамо пар координата **(x, y)**;
- чувамо бројилац и именилац **разломка**;
- **чувамо резултат и додатну информацију**;
- желимо да групишемо неколико вредности без дефинисања посебне класе или структуре.

На пример, у алгоритмима можемо вратити најмањи и највећи елемент низа као торку, уместо да пишемо две засебне функције.

Напомена:

Торка није исто што и низ:

- Низ обично садржи елементе истог типа и приступа им се помоћу индекса који се одређују током извршавања програма.
- Торка има унапред одређен број елемената, а сваки елемент може имати различит тип. У C#-у се, поред тога, именовањем елемената торке може јасније описати значење сваке вредности.

Литература

Детаљније (извори):

- **Курс** на Петљи за **1. разред** гимназијског одељења: **Тема 9.1. Структуре**.
- **Збирка** на Петљи: **Методичка збирка задатака из основа програмирања**: **Структурни тип**.
- **Методичка збирка задатака из основа програмирања**: Задатак: **Разломци**.
- Википедија: **Еуклидов алгоритам**.
- Обавезно пажљиво прочитати лекцију **Еуклидов алгоритам** у оквиру **теме 5.5 Дељивост** на **курсу на Петљи за 1. разред**.
- Препорука је и да се пажљиво прочита лекција **Еуклидов алгоритам**.
- **Збирка** на Петљи: **Методичка збирка задатака из основа програмирања**: **Торке**.
- GitHub: Дељивост: НЗД и Еуклидов алгоритам.
- GitHub: Торке: Разломак.
- GitHub: **Програмски код у C#**.
- GitHub: **Програмски код у C++**.
- Арена: Задатак: Разломци.
- Арена: Структурни тип: Задаци за вежбање: **R2_T09_Strukture_21**.